

FACULDADE DE TECNOLOGIA JOSÉ CRESPO GONZALES
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

ISADORA CORREA DE CARVALHO

MIKE WILLY FRANGUELL

RONALDO ARAUJO SILVA JUNIOR

PROGRAMAÇÃO MULTIPLATAFORMA:
SISTEMA DE MONITORAMENTO AMBIENTAL

Sorocaba - SP

2026

ISADORA CORREA DE CARVALHO

MIKE WILLY FRANGUELL

RONALDO ARAUJO SILVA JUNIOR

**PROGRAMAÇÃO MULTIPLATAFORMA:
SISTEMA DE MONITORAMENTO AMBIENTAL**

Atividade apresentada como critério
avaliativo pelo professor ministrante da
matéria de Programação Multiplataforma.

Orientador: André Cassulino Araújo Souza.

Sorocaba - SP

2026

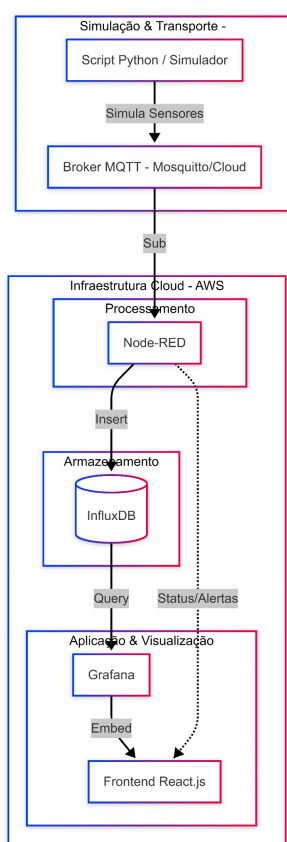
SUMÁRIO

Descrição do Projeto.....	3
Arquitetura.....	3
Explicação da Stack MING.....	4
Explicação da Camada de Aplicação.....	5
Como executar o projeto.....	5
IoT.....	6
Código do ESP32 (Wokwi).....	6
Explicação dos dados enviados.....	7
Node-RED.....	8
Explicação do pipeline.....	9
Backend.....	10
Explicação das regras de negócio.....	10
Frontend.....	11
Código funcional mínimo (exemplo de integração).....	11
Integração com API.....	11

Descrição do Projeto

O projeto consiste em uma solução de monitoramento ambiental. O sistema monitora **temperatura, umidade, CO₂ e luminosidade**, permitindo o acompanhamento em tempo real e a gestão de alertas baseados em condições ambientais específicas.

Arquitetura



A arquitetura segue o modelo de camadas para IoT:

1. **Transporte:** Protocolo MQTT simulando sensores via broker configurado (Google Colab/Cloud).
2. **Processamento:** Node-RED como motor de regras e pipeline.
3. **Armazenamento:** InfluxDB para guardar os dados de ambiente.
4. **Visualização:** Grafana integrado ao Frontend React.
5. **Infraestrutura:** hospedagem e orquestração via **AWS**.

Explicação da Stack MING

A stack **MING** é o conjunto de ferramentas otimizadas para IoT e monitoramento:

- **M (MQTT)**: protocolo de transporte leve para mensagens entre sensores e servidor.
- **I (InfluxDB)**: banco de dados temporal especializado em métricas com alta frequência de escrita.
- **N (Node-RED)**: ferramenta de programação visual para conectar fluxos de dados e lógica de backend.
- **G (Grafana)**: plataforma de análise e visualização de dados via dashboards interativos.

Explicação da Camada de Aplicação

A camada de aplicação é composta pelo **frontend em React + JavaScript**. Ela consome as visualizações do Grafana (via Embed ou API) para exibir os dados de forma amigável ao usuário final, além de gerenciar a interface de status do sistema e o painel de alertas gerados pelo processamento de dados.

Como executar o projeto

1. **Simulador:** iniciar o código no COLAB para começar o envio de dados via MQTT.
2. **Backend/Pipeline:** iniciar a instância do Node-RED e importar o fluxo JSON.
3. **Banco de Dados:** garantir que o bucket no InfluxDB esteja criado e o token configurado no Node-RED.
4. **Frontend:** No diretório do projeto, executar **npm install** e **npm start**.
5. **Visualização:** acessar a aplicação via navegador (URL da AWS).

IoT

Código do ESP32 (Wokwi)

```
1  !pip install paho-mqtt
2
3  import json
4  import time
5  import random
6  from datetime import datetime
7  import paho.mqtt.client as mqtt
8
9  # Configurações MQTT
10 MQTT_SERVER = "3.239.93.13"
11 MQTT_PORT = 1883
12 MQTT_TOPIC = "sensor/Lab01/telemetry"
13
14 # 1. Use a versão da API mais recente se possível, mas mantendo compatibilidade
15 client = mqtt.Client()
16
17 # 2. Conectar
18 client.connect(MQTT_SERVER, MQTT_PORT, 60)
19
20 # 3. IMPORTANTE: Iniciar o loop em segundo plano para manter a conexão viva
21 client.loop_start()
22
23 def get_timestamp():
24     return datetime.now().isoformat()
25
26 def build_json(t, h, lux, co2):
27     data = {
28         "temperatura": round(t, 2),
29         "umidade": round(h, 2),
30         "luminosidade": round(lux, 2),
31         "co2": round(co2, 2),
32         "timestamp": get_timestamp()
33     }
34     return json.dumps(data)
35
36 while True:
37
38     temperatura = random.uniform(20,30)
39     umidade = random.uniform(40,80)
40     luminosidade = random.uniform(100,800)
41     co2 = random.uniform(100, 8000)
42
43     payload = build_json(temperatura, umidade, luminosidade, co2)
44
45     client.publish(MQTT_TOPIC, payload)
46
47     print("Enviado:", payload)
48
49     time.sleep(5)
```

Explicação dos dados enviados.

Os dados são formatados em **JSON** para facilitar o parse:

- **temp**: graus Celsius (°C).
- **hum**: umidade relativa (%).
- **CO₂**: concentração em partes por milhão (ppm).
- **lum**: Nível de luminosidade (lux).

Node-RED

```

1  "processedMetrics": [
2    {
3      "label": "Media hora (temp.)",
4      "value": "25.6 °C",
5      "description": "Agregacao por janela para apoiar analise e reduzir granularidade."
6    },
7    {
8      "label": "Classificacao atual",
9      "value": "CRITICO",
10     "description": "Motor de classificacao simples com estados normal, alerta e critico."
11   },
12   {
13     "label": "Anomalias 24h",
14     "value": "27",
15     "description": "Leituras fora da faixa configurada identificadas automaticamente."
16   },
17   {
18     "label": "Consolidacao",
19     "value": "5 min",
20     "description": "Dados resumidos antes de persistir no banco relacional."
21   }
22 ],
23 "summary": {
24   "averageTemperatureHour": 25.6,
25   "averageHumidityHour": 61.8,
26   "averageCo2Hour": 3439,
27   "anomalyCount": 27,
28   "criticalCount": 25,
29   "latestClassification": "critico"
30 },
31 "services": [
32   {
33     "name": "Broker MQTT",
34     "status": "online",
35     "latency": "18ms",
36     "detail": "Mosquitto em EC2"
37   },
38   {
39     "name": "Node-RED",
40     "status": "online",
41     "latency": "25ms",
42     "detail": "Orquestracao do pipeline"
43   },
44   {
45     "name": "InfluxDB",
46     "status": "online",
47     "latency": "11ms",
48     "detail": "Series temporais"
49   },
50   {
51     "name": "API Node.js",
52     "status": "online",
53     "latency": "31ms",
54     "detail": "REST e regras de negocio"
55   },
56   {
57     "name": "MySQL",
58     "status": "online",
59     "latency": "20ms",
60     "detail": "Consolidacao relacional"
61   },
62   {
63     "name": "Grafana",
64     "status": "online",
65     "latency": "16ms",
66     "detail": "Dashboard operacional"
67   }
68 ],
69 "pipelineSteps": [
70   "CoLab ou ESP32 publica JSON no topico sensor/Lab01/telemetria",
71   "Broker MQTT recebe e encaminha para o Node-RED",
72   "Node-RED valida payload, grava no InfluxDB e chama a API",
73   "Backend Node.js aplica regras e consolida no MySQL",
74   "Frontend React e Grafana exibem dados em tempo real"
75 ]
76 }

```

Explicação do pipeline.

O fluxo recebe as mensagens do broker MQTT no tópico **sensor/lab01/telemetria**. O nó de função converte o payload JSON em um formato compatível com o InfluxDB (Field Set). O pipeline então direciona esses dados para o nó **InfluxDB Out** para persistência e, simultaneamente, avalia regras (ex.: se $\text{CO}_2 > 800$) para disparar notificações de alerta.

Backend

```
1 // Nó: "Processar Dados e Alertas"
2 // Este código recebe o JSON do MQTT e prepara para o InfluxDB
3 // Além de verificar as regras de negócio de CO2
4
5 // 1. Parse do payload recebido do MQTT
6 var dados = msg.payload;
7
8 // 2. Definição do status de alerta para CO2 (Regra de Negócio)
9 if (dados.co2 > 1000) {
10   msg.alert = "CRÍTICO: Nível de CO2 elevado!";
11   msg.status_color = "red";
12 } else {
13   msg.alert = "Normal";
14   msg.status_color = "green";
15 }
16
17 // 3. Formatação para o InfluxDB
18 // Organizamos os campos (fields) e as etiquetas (tags)
19 msg.payload = {
20   temp: dados.temp,
21   hum: dados.hum,
22   co2: dados.co2,
23   lum: dados.lum,
24 };
25
26 // 4. Definição do tópico de saída para o dashboard
27 msg.topic = "monitoramento/status";
28
29 return msg;
30
```

Explicação das regras de negócio.

As regras residem no Node-RED e definem o comportamento do sistema:

- **Alerta de Segurança:** se o nível de CO₂ ultrapassar um limite pré-definido, o status do sistema muda para "Crítico".
- **Consolidação:** os dados são agrupados por tempo no InfluxDB para otimizar a performance do frontend.

Frontend

Código funcional mínimo (exemplo de integração).

```

1
2 import React, { useState, useEffect } from 'react';
3 import { Thermometer, Droplets, Wind, Sun } from 'lucide-react';
4 import Sidebar from '@components/dashboard/Sidebar';
5 import MobileNav from '@components/dashboard/MobileNav';
6 import Header from '@components/dashboard/Header';
7 import SensorCard from '@components/dashboard/SensorCard';
8 import AlertsList from '@components/dashboard/AlertsList';
9 import QuickActions from '@components/dashboard/QuickActions';
10 import SystemStatus from '@components/dashboard/SystemStatus';
11 import * as SensorLib from '@Lib/sensorData';
12
13 const sensors = [
14   { key: 'temperature', title: 'Temperatura', icon: Thermometer, color: '#F97316', gradientFrom: '#F97316', gradientTo: '#FB923C', dataKey: 'temperature' },
15   { key: 'humidity', title: 'Umidade', icon: Droplets, color: '#3B82F6', gradientFrom: '#3B82F6', gradientTo: '#60A5FA', dataKey: 'humidity' },
16   { key: 'co2', title: 'CO2', icon: Wind, color: '#22C55E', gradientFrom: '#22C55E', gradientTo: '#4ADE80', dataKey: 'co2' },
17   { key: 'luminosity', title: 'Luminosidade', icon: Sun, color: '#EAB398', gradientFrom: '#EAB398', gradientTo: '#FACCD5', dataKey: 'luminosity' },
18 ];
19
20 export default function Dashboard() {
21   const [darkMode, setDarkMode] = useState(false);
22   const [isConnected, setIsConnected] = useState(true);
23   const [timeSeriesData, setTimeSeriesData] = useState([]);
24   const [barData, setBarData] = useState([]);
25
26   const get_current_values = () => SensorLib.get_current_values ? SensorLib.get_current_values() : {
27     temperature: { value: 0, unit: '°C' }, humidity: { value: 0, unit: '%' }, co2: { value: 0, unit: 'ppm' }, luminosity: { value: 0, unit: 'Lux' }
28   };
29
30   const [currentValues, set_current_values] = useState(get_current_values());
31   const alerts = SensorLib.get_alerts ? SensorLib.get_alerts() : [];
32
33   useEffect(() => {
34     // Inicializa o gráfico com 30 pontos (janela de 2.5 minutos com steps de 5s)
35     if (SensorLib.generate_time_series_data) setTimeSeriesData(SensorLib.generate_time_series_data(30));
36     if (SensorLib.generate_bar_data) setBarData(SensorLib.generate_bar_data());
37   }, []);

```

Integração com API

A integração ocorre de duas formas:

1. **Dados Históricos/Gráficos:** via incorporação de painéis do Grafana que consultam o InfluxDB.
2. **Status em Tempo Real:** via WebSockets ou requisições HTTP para o endpoint de API exposto pelo Node-RED na AWS.